Kafka Replication and Durability

making acknowledged events survive broker loss

Phase 4 videos on YouTube

In one line: each partition has a leader and follower replicas; the in-sync replica set (ISR) holds followers that are caught up, and with replication factor 3, min.insync.replicas 2, acks=all and unclean leader election off, an acknowledged write survives the loss of any one broker.

1 Acknowledged, then gone

Your Kafka cluster loses a broker, and with it the last few seconds of payment events, events the producers had been told were saved. Kafka can be extremely durable, but only if replication is configured for it.

2 Leaders, followers and the ISR

Each partition has a **replication factor** (RF): the number of copies. One copy is the **leader**, which handles writes (and normally reads); the others are **followers** that fetch from the leader. Followers that are caught up within `replica.lag.time.max.ms` form the **in-sync replica set** (ISR). If the leader fails, the controller elects a new leader from the ISR, so it already has every committed record.

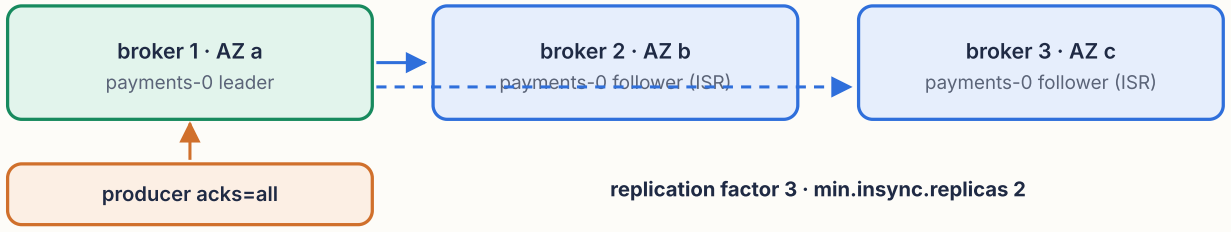


Figure 1. One partition, three replicas in three zones. The leader takes writes; followers copy them.

3 When is a write durable?

With `acks=all`, the leader acknowledges only after every replica currently in the ISR has the record. But if the ISR shrinks to just the leader, 'all' means one copy. `min.insync.replicas` sets a floor: with 2, a write is rejected (`NotEnoughReplicas`) rather than acknowledged with a single copy. **Unclean leader election** lets an out-of-sync replica become leader to restore availability, losing records; keep it off for data that matters.

replication.factor = 3	three copies, one per zone
min.insync.replicas = 2	acks=all needs at least 2 copies
acks = all	producer waits for the in-sync replicas
unclean.leader.election = false	never elect an out-of-date replica
rack awareness	spread replicas across availability zones

Figure 2. The durable configuration.

WORKED EXAMPLE: THE PAYMENTS TOPIC

Replication factor 3, one replica in each of three availability zones (rack awareness), `min.insync.replicas=2`, producers with `acks=all` and idempotence, unclean leader election disabled. Zone B goes down: its follower drops out of the ISR, two copies remain, writes continue. Then the leader in zone A crashes: the follower in zone C, which is in sync, becomes leader with every acknowledged record. Writes pause (one replica left, below the minimum of 2), but nothing acknowledged is ever lost.



Figure 3. What happens as failures pile up with RF 3 and min.insync.replicas 2.

4 Other durability details

- Kafka acknowledges after writing to the OS page cache, not after fsync; replication across zones is what protects you against machine loss.
- Under-replicated partitions are the key alert: they mean you're one failure away from unavailability.
- Cross-region disaster recovery uses MirrorMaker 2 or cluster linking, which is asynchronous.

COMMON MISTAKES

- RF 3 but min.insync.replicas 1, so acks=all can mean one copy.
- All replicas in one availability zone.
- Unclean leader election turned on for financial data.
- Not alerting on under-replicated partitions or a shrinking ISR.

5 Key terms

Replication factor: number of copies of a partition

Leader / follower: the writable copy and the copies that fetch from it

ISR: replicas that are caught up

min.insync.replicas: minimum ISR size for acks=all writes

Unclean leader election: electing an out-of-sync replica, losing data

Under-replicated partition: ISR smaller than RF

6 Interview questions

Q1. What is the ISR? BASIC

The in-sync replicas: the leader plus the followers that have caught up recently. Only ISR members can be elected leader in a clean election.

Q2. Which settings make Kafka writes durable? BASIC

Replication factor 3, min.insync.replicas 2, producer acks=all, unclean leader election off, and replicas spread across zones.

Q3. With RF 3 and min ISR 2, two brokers die. What happens? SCENARIO

Writes to affected partitions fail with NotEnoughReplicas until a replica returns; reads of committed data still work from the remaining replica. No acknowledged data is lost.

Q4. Acknowledged events vanished after a broker crash. What went wrong? SCENARIO

Probably acks=1 or 0, min.insync.replicas 1 with a shrunk ISR, or unclean leader election promoting a lagging replica.

Q5. Does Kafka fsync every write? TRICKY

Not by default; it relies on replication to several machines for durability, with flushing left to the OS. Correlated power loss across all replicas is the residual risk.

Q6. Why not set min.insync.replicas equal to RF? TRICKY

Then any single broker outage or slow follower blocks all writes; RF minus one balances durability and availability.

Next: SD-I-007 · Ordering Guarantees backendroadmap.com